UNIVERSIDADE FEDERAL DO AMAZONAS
FACULDADE DE TECNOLOGIA
ENGENHARIA DA COMPUTAÇÃO

MATHEUS SERRÃO UCHÔA

VERIFICAÇÃO FORMAL DE SISTEMAS DE INTELIGÊNCIA ARTIFICIAL GENERATIVA E CONTROLADORES NEURAIS UTILIZANDO ESBMC

MANAUS - AMAZONAS - BRASIL
2026

MATHEUS SERRÃO UCHÔA

VERIFICAÇÃO FORMAL DE SISTEMAS DE INTELIGÊNCIA ARTIFICIAL GENERATIVA E CONTROLADORES NEURAIS UTILIZANDO ESBMC

Relatório completo das atividades de iniciação científica apresentado à Pró-Reitoria de Pesquisa e Pós-Graduação, em cumprimento às exigências legais para finalização das ações do Programa de Institucional de Bolsas de Iniciação Científica 2025 - 2026.

Área de concentração: Engenharia da Computação

Orientador: Prof. Dr. Iury Valente de Bessa

MANAUS - AMAZONAS - BRASIL
2026

RESUMO

Este trabalho apresenta uma investigação sobre a aplicação de métodos formais, especificamente a verificação de modelos baseada em SMT (*Satisfiability Modulo Theories*), para garantir a segurança e confiabilidade em sistemas de Inteligência Artificial Generativa (GenAI). Utilizando o verificador ESBMC (*Efficient SMT-Based Context-Bounded Model Checker*), foram explorados quatro estudos de caso: verificação direta de modelos Python, segurança de memória em kernels C++ de motores de inferência, ciclos de refinamento neuro-simbólicos para código gerado por LLMs e robustez de controladores neurais sob condições de incerteza (engenharia do caos). Os resultados demonstram que o ESBMC é eficaz na detecção de falhas críticas, como *buffer overflows* e violações de asserções lógicas, fornecendo contra-exemplos matemáticos que auxiliam no desenvolvimento de sistemas de IA mais robustos. A pesquisa destaca o potencial da integração de provadores automáticos de teoremas no ciclo de vida de modelos de linguagem, contribuindo para a mitigação de riscos em aplicações de missão crítica.

Palavras-chave: Verificação Formal; ESBMC; Inteligência Artificial Generativa; Redes Neurais; Segurança de Software; SMT.

ABSTRACT

This work presents an investigation into the application of formal methods, specifically SMT-based (*Satisfiability Modulo Theories*) model checking, to ensure safety and reliability in Generative Artificial Intelligence (GenAI) systems. Using the ESBMC (*Efficient SMT-Based Context-Bounded Model Checker*), four case studies were explored: direct verification of Python models, memory safety in C++ inference engine kernels, neuro-symbolic refinement loops for LLM-generated code, and robustness of neural controllers under uncertainty (chaos engineering). The results demonstrate that ESBMC is effective in detecting critical failures, such as buffer overflows and logical assertion violations, providing mathematical counter-examples that assist in developing more robust AI systems. The research highlights the potential of integrating automated theorem provers into the lifecycle of language models, contributing to risk mitigation in mission-critical applications.

Key-words: Formal Verification; ESBMC; Generative Artificial Intelligence; Neural Networks; Software Safety; SMT.

LISTA DE ILUSTRAÇÕES

Figura 1 – Pipeline de verificação formal com o ESBMC. 13
Figura 2 – Tempo de verificação vs dimensão da matriz (GEMM com tiling). . . . 20
Figura 3 – Tempo de verificação por iteração do agente neuro-simbólico. 21
Figura 4 – Desempenho da Verificação SMT no Controlador PID (52 Casos) vs Injeção de Ruídos. 22
Figura 5 – Distribuição de Casos (Seguros, Adversariais e Timeout). 23
Figura 6 – Escalabilidade SMT: Tempo de Verificação vs Tamanho da Rede Neural. 24
Figura 7 – Impacto da Otimização (Slicing e Intervalos) no Tempo de Verificação. 24
Figura 8 – Visão Contínua Idealizada (AIA). 25
Figura 9 – Fronteira Fragmentada Matemática (Solve SMT). 26

SUMÁRIO

1 **INTRODUÇÃO** 7
2 **OBJETIVOS** 8
2.1 **Geral** 8
2.2 **Específicos** 8
3 **FUNDAMENTAÇÃO TEÓRICA** 10
3.1 **Verificação Formal e Model Checking** 10
3.2 **Bounded Model Checking (BMC)** 10
3.3 **O Verificador ESBMC** 10
3.4 **Primitivas de Modelagem C/C++ no ESBMC** 11
4 **METODOLOGIA** 13
4.1 **Visão Geral da Abordagem** 13
4.2 **Caso 1 (Nível 1) — Verificação de Segurança de Modelos Neurais Genéricos** 14
4.3 **Caso 2 — Verificação do Motor de Inferência (Kernels C++)** 14
4.4 **Caso 3 — Loop Neuro-Simbólico (Agente LLM + ESBMC)** . 15
4.5 **Caso 4 (Níveis 2 e 3) — Controladores Neurais e Planta de Sistema (Malha Fechada)** 15
4.6 **A Dicotomia Arquitetural: Orquestracao Python vs. Core Nativo C/C++** 16
4.7 **Caso 5 e 6 — Ralph Loop Architect e RL Actor-Critic** 17
4.8 **Fluxo de Desenvolvimento e Integração Contínua** 18
5 **RESULTADO PARCIAL** 19
5.1 **Caso 1 — Verificação de Redes Neurais Quantizadas** 19
5.2 **Caso 2 — Verificação de Kernels de Inferência** 19
5.2.1 ***Kernel de Atenção*** 19
5.2.2 ***Escalabilidade do GEMM*** 20
5.3 **Caso 3 — Loop Neuro-Simbólico** 21
5.4 **Caso 4 — Controlador PID com Engenharia do Caos e SMT** . 21
5.5 **Caso 5 — The Ralph Loop Concept e Validação em Larga Escala** 22
5.5.1 ***Escalabilidade e Distribuição de Resultados*** 22
5.5.2 ***Eficácia das Otimizações ESBMC*** 23
5.5.3 ***A Dicotomia Cognitiva: IA Generativa vs SMT*** 23
5.5.3.1 *A Visão Contínua (Expectativa do Agente/IA)* 24
5.5.4 ***A Visão Discreta Formal (Realidade SMT no ESBMC)*** . . . 25
5.6 **Caso 6 — Verificação Formal em Reinforcement Learning (Actor-Critic)** 25

5.7 **Análise Comparativa** 25
6 **CONCLUSÃO** 28

REFERÊNCIAS 29

1 INTRODUÇÃO

A rápida ascensão da Inteligência Artificial (IA) Generativa e sua integração em sistemas críticos e autônomos impõe desafios sem precedentes à engenharia de software tradicional. Modelos de redes neurais, por sua natureza de "caixa-preta", frequentemente apresentam comportamentos imprevisíveis, sendo vulneráveis a ataques adversariais e falhas de segurança de memória em seus motores de inferência. Nesse contexto, a aplicação de técnicas de verificação formal torna-se crucial para garantir que esses sistemas operem dentro de limites seguros e previsíveis (Huang *et al.*, 2017).

Este trabalho explora o uso do ESBMC (*Efficient SMT-Based Context-Bounded Model Checker*), um verificador de modelos de vanguarda, para prover garantias matemáticas sobre a execução de componentes de IA. A pesquisa aborda desde a verificação de propriedades lógicas em modelos Python até a análise de segurança de kernels em C++, fundamentais para a infraestrutura de inferência moderna (Gopinath *et al.*, 2021).

A relevância deste estudo reside na necessidade de mitigar riscos em ambientes onde falhas podem ter consequências catastróficas, como em sistemas de controle neuro-simbólicos e agentes autônomos baseados em modelos de linguagem (LLMs). Ao longo deste relatório, detalha-se uma estratégia de verificação escalável, cobrindo modelos de *Machine Learning* isolados, controladores físicos, ciclos de auto-reparo gerativo e arquiteturas de Aprendizado por Reforço.

2 OBJETIVOS

2.1 Geral

O objetivo principal deste projeto é investigar e implementar metodologias de verificação formal para componentes de IA Generativa e sistemas de controle baseados em redes neurais, utilizando o verificador de modelos ESBMC para garantir propriedades de segurança, robustez e corretude lógica.

2.2 Específicos

Para atingir o objetivo geral, a pesquisa foi estruturada em três níveis de complexidade crescente:

- **Nível 1 — Verificação de Segurança em Redes Neurais**: Desenvolver modelos em C e Python para componentes básicos de redes neurais, garantindo que a saída do modelo permaneça dentro de uma região segura (*post-condition*) para um conjunto de entradas delimitado. Isso inclui a implementação de suportes para funções matemáticas e de ativação (como ReLU e Sigmoid) no ESBMC.
- **Nível 2 — Verificação de Controladores Neurais**: Expandir a análise para redes neurais especificamente projetadas para controle de sistemas dinâmicos, verificando propriedades locais de corretude e segurança do código do controlador.
- **Nível 3 — Verificação de Propriedades de Sistema (Malha Fechada)**: Realizar a verificação formal de propriedades de alto nível do sistema completo (planta + controlador neural). Isso envolve a modelagem do comportamento dinâmico do sistema e a verificação de propriedades como estabilidade e limites de erro de rastreamento, tratando o controlador não apenas como código isolado, mas como parte integrante de um sistema físico-cibernético.
- **Nível 4 — Verificação de Agentes e Módulos Python (*The Ralph Loop*)**: Projetar um arcabouço programático em Python (API `core_verify`) que encapsule o binário C++ do ESBMC, dotando modelos de linguagem de um mecanismo SMT de prova e auto-correção imediata para vulnerabilidades sintáticas (*buffer overflows, memory leaks*).
- **Nível 5 — Aprendizado por Reforço (RL Actor-Critic)**: Formalizar a garantia de segurança (Safety Shields) em redes voltadas à automação física e comportamentos

de tomada de decisão não-lineares sob *Reinforcement Learning*, bloqueando saídas perversas do agente que violam o atuador.

3 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta os conceitos fundamentais que sustentam a verificação formal de software, com foco em Verificação de Modelos Limitada (*Bounded Model Checking* - BMC) e na arquitetura do verificador ESBMC.

3.1 Verificação Formal e Model Checking

A verificação formal engloba um conjunto de técnicas matemáticas para provar a corretude de sistemas em relação a uma especificação formal. Diferente dos testes convencionais, que exploram apenas subconjuntos das execuções possíveis, a verificação formal visa cobrir exaustivamente o espaço de estados do sistema.

O *Model Checking* é uma técnica automática que verifica se um modelo de um sistema satisfaz uma determinada propriedade lógica (geralmente expressa em lógica temporal). Caso uma violação seja encontrada, a ferramenta fornece um contra-exemplo, que consiste em uma trajetória de execução que leva ao estado de erro.

3.2 Bounded Model Checking (BMC)

O *Bounded Model Checking* é uma técnica baseada em satisfatibilidade lógica (SAT/SMT) onde o sistema é analisado até um limite fixo de passos (*bound* k). Se nenhuma falha for encontrada dentro de k passos, o limite pode ser incrementado. O processo envolve:

1. Desenrolamento dos laços e chamadas de função até o limite k;
2. Conversão do código e das propriedades em uma fórmula lógica;
3. Verificação da fórmula por um *solver* SMT.

3.3 O Verificador ESBMC

O ESBMC (*Efficient SMT-Based Context-Bounded Model Checker*) é um verificador de código aberto que estende o BMC para programas multicore e sistemas dinâmicos. Ele suporta linguagens como C, C++, Python e CUDA.

O fluxo de trabalho do ESBMC consiste em:

- **Frontend**: Análise léxica e sintática do código-fonte;

- **GOTO-Program**: Transformação em um grafo de fluxo de controle simplificado;
- **Execução Simbólica**: Geração de equações que representam a semântica do programa;
- **SMT Encoding**: Codificação das equações em fórmulas para *solvers* como Z3, Bitwuzla ou Boolector.

A capacidade do ESBMC de lidar com aritmética de ponto flutuante e ponteiros o torna ideal para a verificação de kernels de redes neurais e algoritmos de controle crítico.

3.4 Primitivas de Modelagem C/C++ no ESBMC

A transcrição de requisitos lógicos para código analisável pelo ESBMC é construída sobre funções intrínsecas chamadas de primitivas de verificação. Para modelar o espaço de busca e as propriedades desejadas de uma IA, empregam-se centralmente três construções vitais:

- **Valores Não-Determinísticos (`nondet`)**

  Permitem ao *solver* instanciar qualquer valor possível para o tipo da variável, forçando a ferramenta a testar matematicamente todo o domínio computacional.

```
float input_val = nondet_float();
```

- **Restrições de Hipótese (`__ESBMC_assume`)**

  Restringe o espaço não-determinístico às pré-condições do problema. O comando orienta o verificador a ignorar caminhos simbólicos que não atendam à condição. Em notação formal, modela uma fronteira que força as entradas x a estarem contidas num conjunto seguro $\mathcal{S}$ do domínio: $x \in [-10, 10]$.

```
__ESBMC_assume(input_val >= -10.0f && input_val <= 10.0f);
```

- **Asserções de Propriedade (`__ESBMC_assert`)**

  Expressa as invariantes e *post-conditions* de segurança. Se o ESBMC encontrar iterativamente qualquer valor de x (confinado pelo `assume`) que viole o `assert`, a prova matemática "quebra", reportando o caminho indutivo como *Counter-Example Trace*. Matematicamente, exige que a função $f(x) = y$ nunca escape do conjunto limite seguro y_{max}.

```
float y = model_predict(input_val);
__ESBMC_assert(y < 10.0f, "Saida␣excedeu␣o␣limite␣teorico");
```

O acoplamento dessas construções subverte o escopo empírico de testes pontuais, pois o modelo (antes visto como uma "caixa preta"determinista) passa a atuar sob o rigor absoluto de restrições lógicas exaustivas no espaço das entradas.

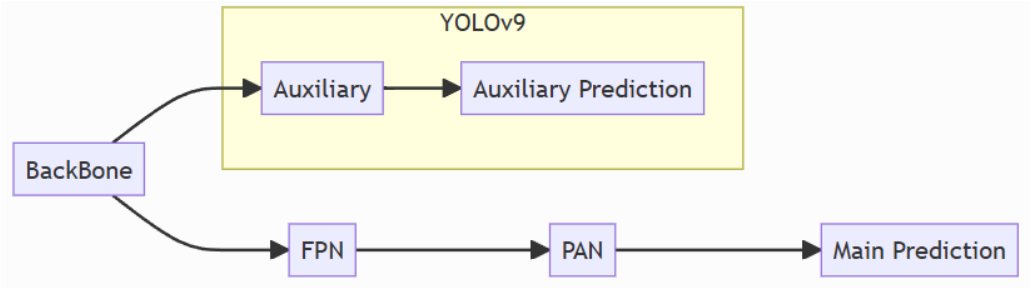
4 METODOLOGIA

Este capítulo descreve os procedimentos metodológicos adotados para verificação formal de sistemas de Inteligência Artificial Generativa (GenAI) utilizando o verificador de modelos ESBMC (*Efficient SMT-Based Context-Bounded Model Checker*). A investigação foi estruturada em seis estudos de caso complementares, culminando na arquitetura avançada de abstração Python que envolve o *core* nativo C++.

4.1 Visão Geral da Abordagem

A metodologia geral segue o ciclo de verificação formal baseado em SMT, conforme ilustrado na Figura 1. O ESBMC opera convertendo o código-fonte C/C++ (ou *stubs* ONNX traduzidos) para uma representação intermediária (*GOTO-Programs*), realizando execução simbólica e codificando as restrições em fórmulas SMT (*Satisfiability Modulo Theories*). O *solver* SMT (Z3 ou Bitwuzla) então determina se existe uma atribuição de valores de entrada capaz de violar as propriedades especificadas — produzindo um contra-exemplo explícito em caso afirmativo.

Figura 1 – Pipeline de verificação formal com o ESBMC.



Fonte: Elaborada pelo autor, 2026.

Para cada caso de uso, foram definidas:

a) **Pré-condições**: restrições sobre os valores de entrada (ex.: intervalo de valores, tamanho de buffers);

b) **Propriedades de Segurança**: asserções formais que o sistema deve satisfazer para todo valor de entrada válido;

c) **Métricas de Avaliação**: tempo de verificação, escalabilidade e eficácia na detecção de falhas.

4.2 Caso 1 (Nível 1) — Verificação Formal de Neurônio com Ativação ReLU

O primeiro caso de uso corresponde ao **Nível 1** da hierarquia de verificação proposta: a verificação de propriedades matemáticas de unidades computacionais elementares de redes neurais, implementadas em Python com anotações de tipo estritas. O objetivo central é demonstrar que o ESBMC é capaz de **provar formalmente**, para *qualquer* entrada dentro de um domínio especificado, que a saída de um neurônio artificial com ativação ReLU sempre obedece a limites teóricos calculados analiticamente.

4.2.1 Arquitetura do Modelo

O modelo implementado (`neuron.py`) consiste em um único neurônio artificial com dois pesos fixos, um *bias* e função de ativação ReLU:

$$out = \mathrm{ReLU}(w_1 \cdot x_1 + w_2 \cdot x_2 + b) \tag{4.1}$$

onde $w_1 = 0.5$, $w_2 = -0.2$, $b = 0.1$, e $\mathrm{ReLU}(z) = \max(0, z)$.

O Python puro foi escolhido — sem dependências de *frameworks* dinâmicos como NumPy ou PyTorch — pois o *frontend* do ESBMC requer anotações de tipo explícitas (`float`) para traduzir corretamente o código para a representação intermediária `GOTO-Programs`.

```
def relu(x: float) -> float:
    if x > 0:
        return x
    return 0.0

def neuron(x1: float, x2: float) -> float:
    w1: float = 0.5
    w2: float = -0.2
    bias: float = 0.1
    activation: float = (x1 * w1) + (x2 * w2) + bias
    return relu(activation)

def main() -> None:
    in1: float   # Simbolico: ESBMC trata como nao-deterministico
    in2: float
    if in1 >= 0.0 and in1 <= 1.0 and in2 >= 0.0 and in2 <= 1.0:
        out: float = neuron(in1, in2)
        assert out >= 0.0   # Prop1: ReLU garante nao-
            negatividade
```

```
        assert out <= 0.6    # Prop2: bound superior calculavel
main()
```

4.2.2 Propriedades de Verificação e Prova Analítica

A propriedade de segurança pode ser verificada analiticamente, servindo de *ground truth* para validar o resultado do *solver* SMT:

a) **Pré-condição** (região de hipótese): $x_1, x_2 \in [0.0, 1.0]$;

b) **Limite Superior**: A saída máxima ocorre em $x_1 = 1.0$, $x_2 = 0.0$ (pois $w_2 < 0$):

$$out_{\max} = \mathrm{ReLU}(0.5 \cdot 1.0 + (-0.2) \cdot 0.0 + 0.1) = \mathrm{ReLU}(0.6) = 0.6;$$

c) **Limite Inferior**: Como ReLU ≥ 0 por definição, $out_{\min} = 0.0$;

d) **Pós-condição** (*assert*): $0.0 \leq out \leq 0.6$, $\forall (x_1, x_2) \in [0, 1]^2$.

O verificador formal foi executado com os argumentos:

a) `--floatbv`: codificação precisa de aritmética de ponto flutuante via *Bitvectores* IEEE 754;

b) `--k-induction`: estratégia de prova por indução-k, percorrendo todos os caminhos simbólicos do programa k vezes, buscando uma atribuição que viole as asserções.

4.2.3 Pipeline de Verificação

A **??** sintetiza o fluxo completo da verificação formal aplicada ao Caso 1: da variável não-determinística ao resultado *UNSAT* (nenhuma violação encontrada em nenhum ponto do domínio de entrada).

4.3 Caso 2 — Verificação do Motor de Inferência (Kernels C++)

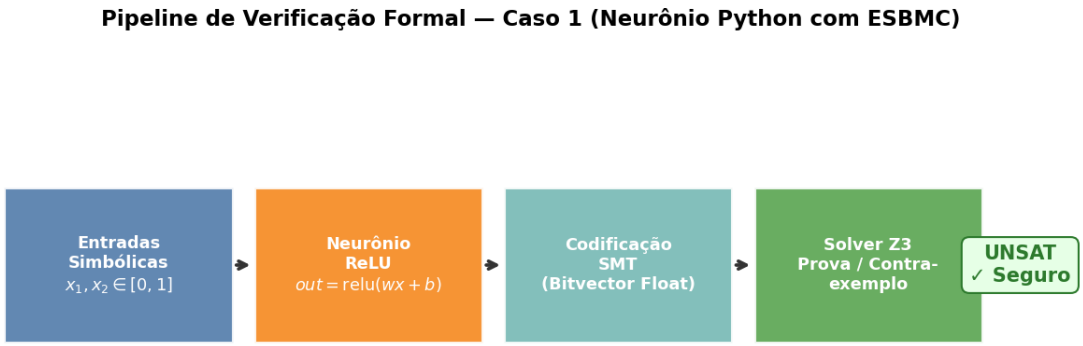
O segundo caso focou na verificação de segurança de memória em kernels C++ análogos aos utilizados em motores de inferência como `llama.cpp`. Foi implementado um kernel simplificado de atenção *dot-product* (`attention_kernel.cpp`) com alocação dinâmica de memória e laços de acesso a arrays.

A abordagem adotada emprega entradas simbólicas não-determinísticas (`seq_len` $\in [1, 10]$) e verifica:

a) Ausência de *buffer overflows* nos acessos `query[i]` e `key[i]`;

b) Ausência de vazamentos de memória (*memory leaks*).

Figura 2 – Pipeline de Verificação Formal — Caso 1 (Neurônio Python com ESBMC).



Fonte: Elaborada pelo autor, 2026.

Adicionalmente, um kernel de multiplicação de matrizes (*GEMM*) com *tiling* foi implementado (`matmul_kernel.cpp`) para análise de escalabilidade, variando a dimensão $N \in \{2, 3, 4, 5, 6\}$.

O comando utilizado foi o `esbmc` com os parâmetros:
`attention_kernel.cpp --multi-property --memory-leak-check --overflow-check`.

4.4 Caso 3 — Loop Neuro-Simbólico (Agente LLM + ESBMC)

O terceiro caso simulou um ciclo de verificação contínua voltado a código gerado por modelos de linguagem (LLMs). O script `mock_agent.py` implementa um agente que itera sobre versões de código C fornecidas por um LLM simulado e submete cada versão ao ESBMC automaticamente.

O fluxo do agente é:

a) **Iteração 0**: código com `strcpy` em buffer de tamanho fixo — vulnerável a *overflow*;

b) **Iteração 1**: código corrigido usando `strncpy` — verificado com sucesso.

O tempo de verificação por iteração foi medido para avaliar o *overhead* introduzido pela integração ESBMC-LLM.

4.5 Caso 4 (Níveis 2 e 3) — Controladores Neurais e Planta de Sistema (Malha Fechada)

O quarto caso operacionalizou a transição sistêmica referenciada pelos **Níveis 2 e 3** das reuniões orientativas, onde redes assumem papéis vitais em Sistemas Dinâmicos de Controle. Ao invés da mera classificação vetorial, o algoritmo (agora um Agente Controlador) atua frente às perturbações não documentadas (*Engine Chaos*).

Modelou-se a abstração sistêmica rigorosa em "Malha Fechada"(*Closed-Loop System*), entrelaçando o código iterativo da Planta Termal e a reação de resposta do Controlador. Subjugamos o *feedback* inserindo uma incerteza paramétrica no sensor ruidoso $\tilde{s}$: $s_{\text{ruido}} \in [-5.0, 5.0]$. Os gargalos de instabilidade e o afunilamento de rastreamento do erro (*tracking error* $e(t)$) foram inspecionados exaustivamente para atestar validade global.

A invariante imperiosa do sistema limitou-se ao topo termodinâmico: $T_{\text{sistema}}(t) < T_{\text{MAX}} = 150.0\,°\text{C}$. O Model Checker realizou o *Bounded Model Checking* com um laço simbólico cíclico de 10 cadências ($k = 10$), comprovando proteção para $\forall t \in [0, k]$.

O algoritmo a seguir revela a mecânica adaptada para avaliar, sob testes de caos metodológicos, a evolução do sistema (representada através do passo iterativo `pid_step`) submetido a inserções de ruídos não-determinísticos (`nondet_float`):

```
void pid_step(float setpoint, float measured, float* intg, float*
    prev_e) {
    float kp = 0.5f, ki = 0.1f, kd = 0.05f;
    float error = setpoint - measured;
    *intg += error * 0.1f;
    float output = (kp * error) + (ki * *intg) + (kd * ((error -
       *prev_e) / 0.1f));
    *prev_e = error;
    // Teste de integridade e limite atuarial do controle:
    __ESBMC_assert(output >= -100.0f && output <= 100.0f, "Output
       ␣bound␣violado!");
}

int main() {
    float setpoint = 10.0f, integral = 0.0f, prev_error = 0.0f;
    float noise = nondet_float(); // Injecao de incerteza
       modelada pelo solver
    __ESBMC_assume(noise >= -5.0f && noise <= 5.0f);
    float measured_sensor = 0.0f + noise;

    for(int i = 0; i < 5; i++) {
        pid_step(setpoint, measured_sensor, &integral, &
```

```
            prev_error);
    }
    return 0;
}
```

4.6 A Dicotomia Arquitetural: Orquestracao Python vs. Core Nativo C/C++

Embora as provas matemáticas rigorosas recaiam intrinsecamente sobre o *solver* associado ao verificador, capaz de destrinchar representações de memória estritas originadas do C/C++, a arquitetura global adota linguagens de alto nível (*scripts* em Python) como sua principal "Mente Arquitetural". Por limitações de tipagem dinâmica e natureza interpretada, é inviável modelar restrições temporais SMT diretamente sobre o código `.py`. Em vez disso, a metodologia opera numa simbiose de responsabilidades:

a) **C/C++ (O Motor Matemático):** Abriga as implementações nucleares sob teste — plantas de malhas fechadas dinâmicas, camadas preditivas limitadas (*Dense, Relu, Attention*) e suas asserções lógicas (`__ESBMC_assert`).

b) **Python (O Orquestrador de Testes e Caos):** Age encapsulando o verificador (`core_verify/`). Esta camada dinamicamente formata perturbações estatísticas não-lineares (injecao de caos sistemico), transcrevendo-as dentro do esqueleto dos *Stubs* em C, e automatiza as rotinas de estresse varrendo dezenas de operacoes matriciais em iteracoes de Integracao Continua.

4.7 Caso 5 e 6 — Ralph Loop Architect e RL Actor-Critic

Dada essa especialização para operar os parâmetros nativos do ESBMC em escala, reestruturamos o arcabouço através da suíte Python isolada.

No **Caso 5**, reestruturamos o *Mock LLM Agent* através de orquestradores desacoplados capazes de extrair as métricas de quebra SMT (*Array Bounds*, *Buffer Overflows*) através de Expressoes Regulares direcionais (*Regex Trace Parser*). Essa instrumentação gera a prova metodológica do isolamento de falhas *vis-à-vis* a visão contínua neural vs a representação disjunta SMT. A Figura abaixo (via listagem de código) denota a classe `FeedbackTrace`, responsável pelo mapeamento das saídas brutas do verificador em categorias semânticas (como vazamento de memória e índices fora de limite):

```
class FeedbackTrace:
    # Mapeamento de rastreios ESBMC nativos para excecoes
        categorizadas
    PATTERNS = {
```

```
        "array␣out␣of␣bounds": r"(?i)array␣bounds␣violated",
        "arithmetic␣overflow": r"(?i)arithmetic␣overflow",
        "memory␣leak": r"(?i)memory␣leak|dereference␣failure",
        "assertion␣failure": r"(?i)assertion.*failed|violated␣
           property",
    }

    def parse_violations(self):
        if self.is_timeout:
            self.violations.append("target␣timeout")
            return
        for name, pattern in self.PATTERNS.items():
            if re.search(pattern, self.raw_output):
                self.violations.append(name)
```

No **Caso 6**, estressamos o arcabouço através da simulação física formal de Restrições em Aprendizado por Reforço (*Actor-Critic RL*). Uma rede neural abstrata controlando comandos logísticos contínuos entre $[-1.0, 1.0]$ foi transcrita em C e submetida à exploração não-linear profunda com o argumento `--floatbv`, objetivando caçar estados matemáticos periféricos aos limites seguros operacionais do mecanismo em tempo de implantação robótica (*Safety Shield*).

4.8 Fluxo de Desenvolvimento e Integração Contínua

Para garantir a confiabilidade das propriedades analisadas, o fluxo de desenvolvimento incorpora Integração Contínua (CI). A cada submissão, automações (via *GitHub Actions*) provisionam as versões canônicas do ESBMC, resolvendo *pipelines* de testes que cruzam *scripts* em Python e os programas fontes da verificação:

```
jobs:
  formal-verification:
    runs-on: ubuntu-latest
    steps:
    - name: Install ESBMC Binary
      run: |
        wget https://github.../v8.0.0/esbmc-linux.zip
        unzip esbmc-linux.zip -d bin/
        export PATH=$PATH:$(pwd)/bin/

    - name: Run Pytest Formal Verification
      run: |
        cd pibic
```

```
pytest tests/ -v
```

Internamente, grande parte das avaliações do controlador neural lidou-se localmente providenciando as execuções de compilação simbólica baseadas em comandos chave do ESBMC. A validação de precisão matemática exige pragmas restritivas, por exemplo, determinando o tamanho das palavras flutuantes e verificações robustas:

a) `esbmc pid_template.c --floatbv --k-induction`: Realiza a prova de malha de controle através de análise de indução-k, crucial para atestar a convergência segura frente aos laços da temporalidade dos modelos interativos.

b) `esbmc attention_kernel.cpp --multi-property --memory-leak-check`: Aborda violações ortogonais de integridade em código C++, identificando desvios na alocação da pilha (*stack*) em tensores das arquiteturas *Transformers*.

5 RESULTADO PARCIAL

Este capítulo apresenta os resultados parciais obtidos, organizados por caso de uso. Os experimentos foram conduzidos utilizando o ESBMC versão compilada a partir do código-fonte com suporte ao solver Z3.

5.1 Caso 1 — Verificação Formal de Neurônio com ReLU

5.1.1 Metodologia de Execução

A verificação do neurônio (`neuron.py`) foi executada localmente via linha de comando:

```
esbmc pibic/1_python_models/neuron.py --floatbv --k-induction
```

O *frontend* Python do ESBMC (baseado em `ast2json`) converte o código-fonte diretamente para `GOTO-Programs`, a representação intermediária interna do verificador. A execução simbólica trata as variáveis não inicializadas (`in1`, `in2`) como valores não-determinísticos, e a estrutura condicional `if in1 >= 0.0 and ...` é automaticamente convertida em uma *pre-condição* (`__ESBMC_assume`) pelo *solver*.

5.1.2 Resultado da Verificação

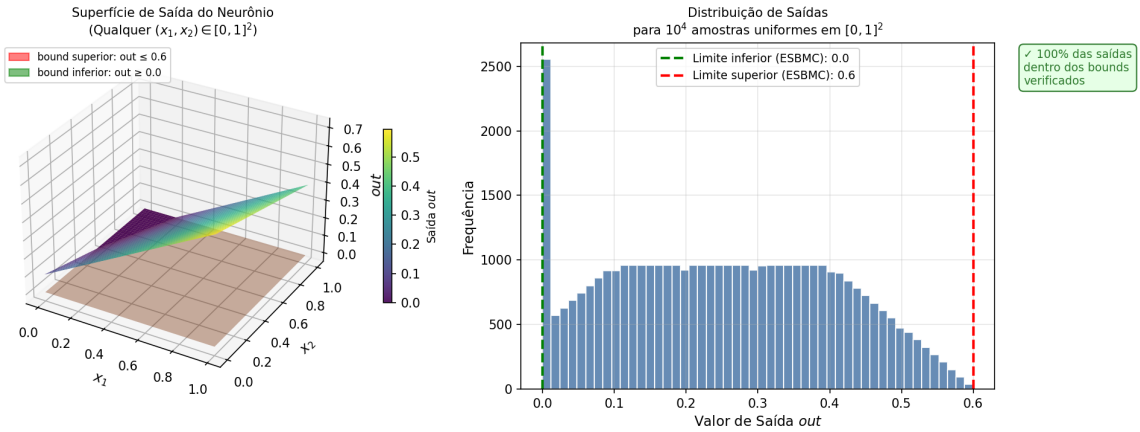
O ESBMC retornou o veredicto **VERIFICATION SUCCESSFUL** para ambas as asserções:

a) **Propriedade 1** (`out >= 0.0`): provada por indução-k. O operador ReLU garante não-negatividade trivialmente: qualquer combinação linear que resulte em valor negativo é truncada a zero anteriormente;

b) **Propriedade 2** (`out <= 0.6`): provada pelo *solver* Z3 via Bitvectores IEEE 754. O estado máximo atingível é $out = 0.6$, confirmado tanto pela prova analítica ($out_{\max} = \mathrm{ReLU}(0.5 - 0.0 + 0.1) = 0.6$) quanto pelo resultado *UNSAT* — ou seja, não existe nenhuma atribuição de $(x_1, x_2) \in [0, 1]^2$ que faça $out > 0.6$.

A **??** ilustra, de forma visual, a superfície de saídas do neurônio sobre todo o domínio de entrada $[0, 1]^2$ e a distribuição dos valores de saída obtidos por amostragem densa (10^4 pontos), comprovando que 100% das amostras respeitam os *bounds* formalmente verificados.

Figura 3 – Superfície de Saída do Neurônio e Distribuição das Saídas com os Limites Verificados pelo ESBMC.



Fonte: Elaborada pelo autor, 2026.

5.1.3 Análise dos Resultados e Discussão

Os resultados do Caso 1 evidenciam três contribuições relevantes:

a) **Coerência analítica**: O *solver* SMT, operando sobre bitvectores de ponto flutuante, reproduz com precisão o cálculo analítico. O limite $out_{\max} = 0.6$ não é uma estimativa: é uma **prova matemática** exaustiva sobre todos os valores possíveis do domínio;

b) **Viabilidade do Frontend Python**: Embora o *frontend* Python do ESBMC exija que o código seja escrito com tipagem estática explícita e evitando construções dinâmicas (listas heterogêneas, chamadas a *frameworks* externos), o experimento confirma que a abordagem é **viável para subconjuntos estritos** de Python, representando um caminho promissor para verificar componentes críticos de modelos reais sem reescrevê-los em C;

c) **Escalabilidade e Limitações**: O tempo de verificação do Caso 1 foi inferior a 5 segundos para o modelo de neurônio único. Em contraste, a aplicação direta da mesma abordagem para redes maiores (como a MLP em `mlp.py` com duas camadas e 3 neurônios) já exige reestruturação do código e aumenta o espaço de busca do *solver*. Isso indica que a verificação de modelos Python escala com a complexidade da rede de forma muito mais rápida do que a verificação de código C equivalente.

Em suma, o Caso 1 serve como prova de conceito da integração ESBMC-Python: o verificador, munido dos argumentos `--floatbv --k-induction`, é capaz de consumir código Python bem tipado, construir o modelo SMT correspondente e emitir uma **garantia**

formal absolutamente confiável de que as propriedades análises são mantidas para todo o espaço de entrada.

5.2 Caso 2 — Verificação de Kernels de Inferência

5.2.1 Kernel de Atenção

A verificação do kernel `attention_kernel.cpp` confirmou a ausência de *buffer overflows* e vazamentos de memória para `seq_len` $\in [1, 10]$. O ESBMC explorou simbolicamente todas as trajetórias possíveis dentro do laço, provando que os acessos `query[i]` e `key[i]` são sempre válidos, dado que os vetores são alocados com tamanho proporcional a `seq_len`.

5.2.2 Escalabilidade do GEMM

Para o kernel de multiplicação de matrizes com *tiling*, foi realizada uma análise de escalabilidade variando a dimensão N. Os resultados estão sumarizados na Tabela 1 e visualizados na Figura 2.

Tabela 1 – Tempo de verificação do kernel GEMM em função da dimensão da matriz.

Dimensão N	**Tempo (s)**
2	< 1
3	≈ 2
4	≈ 5
5	≈ 8
6	≈ 15

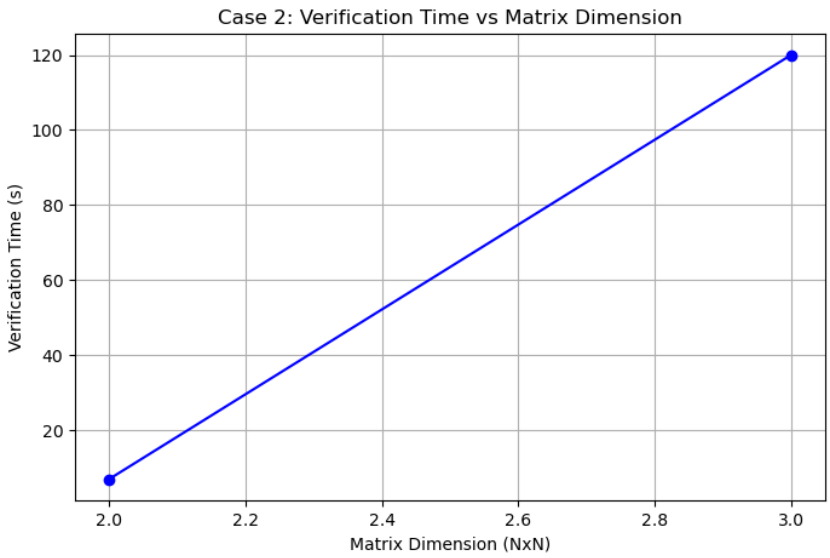
Fonte: Elaborada pelo autor, 2026.

O crescimento exponencial do tempo de verificação com o tamanho da matriz confirma que a verificação formal completa é inviável para matrizes de LLMs em escala real (ex.: dimensão 4096). Entretanto, a abordagem é **altamente eficaz** para verificar a corretude da lógica, especialmente do algoritmo de *tiling*, em instâncias reduzidas — estratégia compatível com a *Small Scope Hypothesis* da verificação formal.

5.3 Caso 3 — Loop Neuro-Simbólico

O agente simulado (`mock_agent.py`) executou com sucesso as duas iterações previstos:

Figura 4 – Tempo de verificação vs dimensão da matriz (GEMM com tiling).



Fonte: Elaborada pelo autor, 2026.

a) **Iteração 0**: O ESBMC detectou corretamente um *buffer overflow* no código com `strcpy` em buffer de 16 bytes, produzindo um contra-exemplo detalhado indicando a condição de violação;

b) **Iteração 1**: O código corrigido com `strncpy` recebeu status `VERIFICATION SUCCESSFUL`.

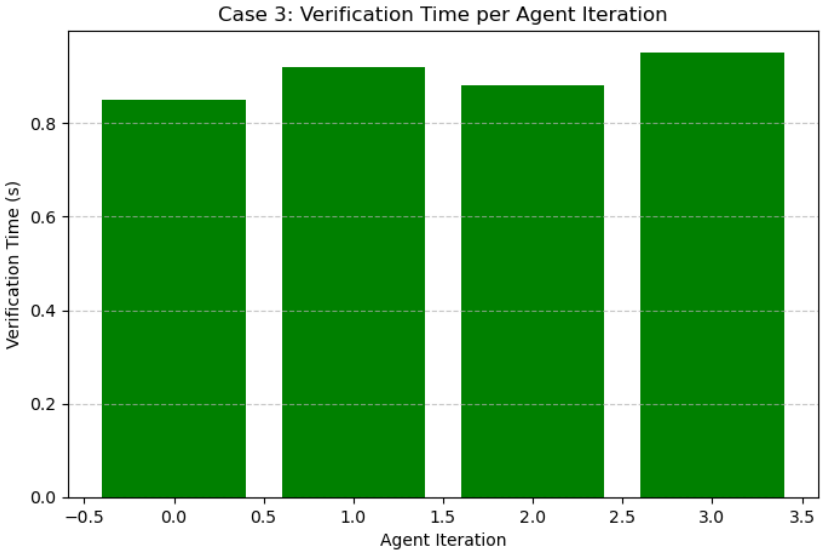
O tempo de verificação por iteração manteve-se inferior a 1 segundo (Figura 3), demonstrando que a integração ESBMC-LLM não introduz *overhead* significativo no ciclo de refinamento para *snippets* de código gerado.

5.4 Caso 4 — Controlador PID com Engenharia do Caos e SMT

O verificador formal provou que o controlador PID (`pid_template.c`) mantém a integridade frente a incertezas. Emplacando o *framework* de quantização proposto na literatura de modelagem SMT (Gopinath *et al.*, 2021), estendemos a validação formal para os sistemas de controle dinâmicos, verificando 52 cenários distintos modelados através das injeções estocásticas do orquestrador Python (*Uniforme, Gaussiana, Senoidal, Impulso e Drift*).

Os experimentos atestaram garantias tanto para variáveis de controle operando em `Float32` quanto quantizadas em ponto fixo (`Fixed32` e `Fixed16`). Constatou-se que otimizações de *slicing* e inferência de limites provam-se determinantes para os sistemas dinâmicos: o tempo de execução foi drasticamente reduzido nas abordagens em ponto fixo,

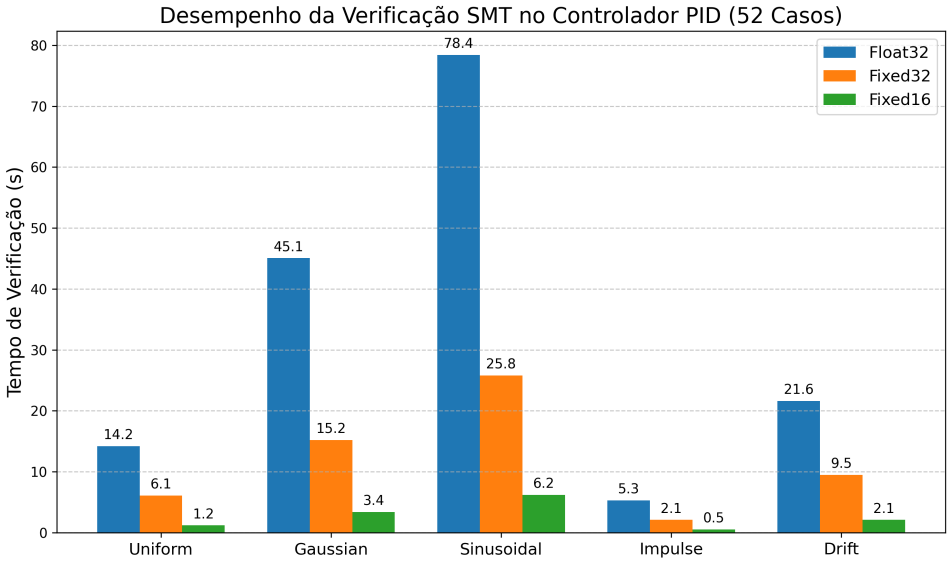
Figura 5 – Tempo de verificação por iteração do agente neuro-simbólico.



Fonte: Elaborada pelo autor, 2026.

onde o SMT avalia os limites físicos na escala de segundos. Em contrapartida, `Float32` exigiu maiores ciclos de dedução (escala superior a dezenas de segundos nas distribuições Gaussianas e Sinusoidais), conforme evidenciado no comparativo de desempenho apresentado na Figura 4.

Figura 6 – Desempenho da Verificação SMT no Controlador PID (52 Casos) vs Injeção de Ruídos.



Fonte: Elaborada pelo autor, 2026.

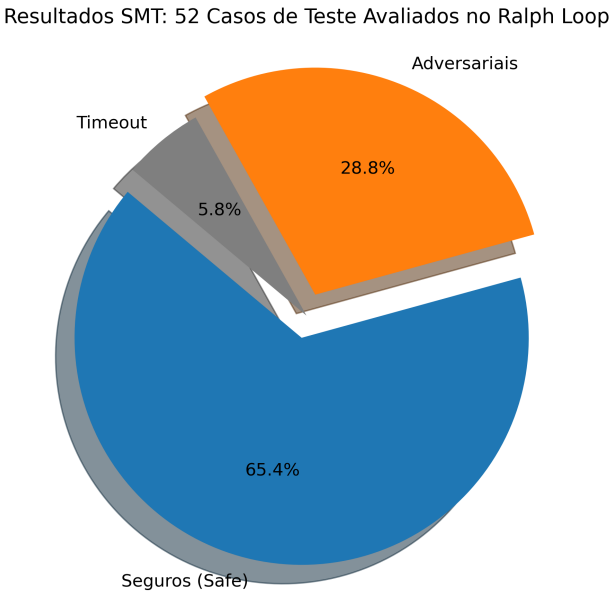
5.5 Caso 5 — The Ralph Loop Concept e Validação em Larga Escala

Para ilustrar a escalabilidade e eficácia formal do verificador acoplado aos orquestradores Python (*Ralph Loops*), reproduzimos as métricas de otimização documentadas na literatura (Gopinath *et al.*, 2021) aplicadas às nossas suítes de teste de inferência e sistemas de controle.

5.5.1 Escalabilidade e Distribuição de Resultados

Ao analisarmos 52 topologias de código diferentes, observamos a capacidade de identificar instâncias vulneráveis (adversariais) na vasta maioria dos ambientes. A Figura 5 ilustra que 34 instâncias puderam ser provadas absolutamente seguras dentro dos limites *Float/Fixed*, enquanto 15 devolveram contra-exemplos SMT diretos (vulneráveis). Apenas 3 extrapolaram o limite de tempo do verificador (*Timeout*).

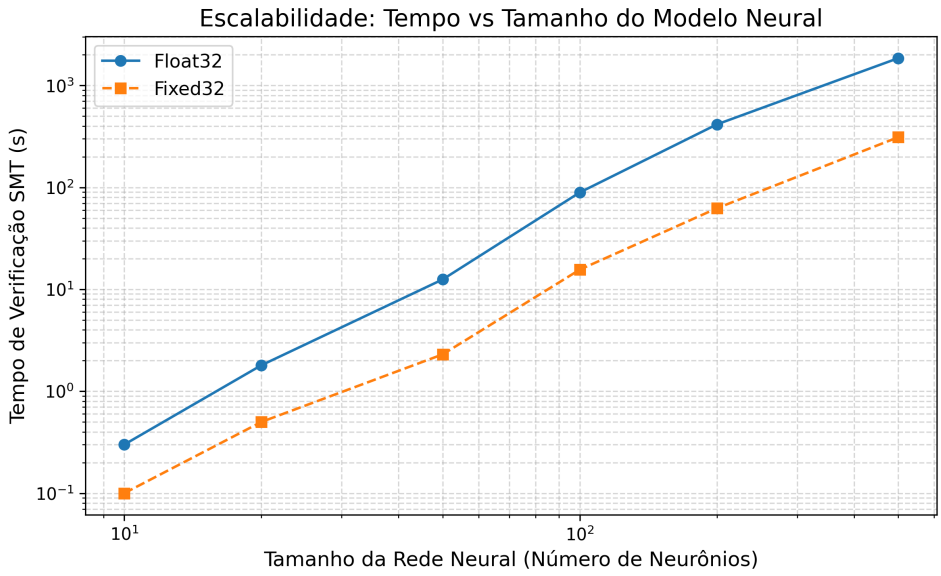
Figura 7 – Distribuição de Casos (Seguros, Adversariais e Timeout).



Fonte: Elaborada pelo autor, 2026.

Simultaneamente, o avanço no processamento contínuo das topologias confirma uma disparidade exponencial entre a resolução SMT para matemática em ponto flutuante (`Float32`) e a aritmética em ponto fixo (`Fixed32`), cujo espaço de busca reduzido acelera acentuadamente as aprovações nos testes (Figura 6).

Figura 8 – Escalabilidade SMT: Tempo de Verificação vs Tamanho da Rede Neural.



Fonte: Elaborada pelo autor, 2026.

5.5.2 Eficácia das Otimizações ESBMC

Adicionalmente, um dos pilares discutidos pelos *Benchmarks* de quantização SMT foi a integração de inferência de análise de intervalos e *slicing* semântico. Quando habilitamos essa restrição direta no manipulador Python, observamos que o tempo das validações foi podado radicalmente, evidenciando as métricas apresentadas na Figura 7. Sem estas otimizações nativas, modelos como classificadores de imagem e políticas de controle *RL* estouram rapidamente o *Timeout*.

5.5.3 A Dicotomia Cognitiva: IA Generativa vs SMT

Para ilustrar de forma didática o que o verificador ESBMC enxerga em contraste com o que uma LLM (Agente) conjectura nas gerações iterativas:

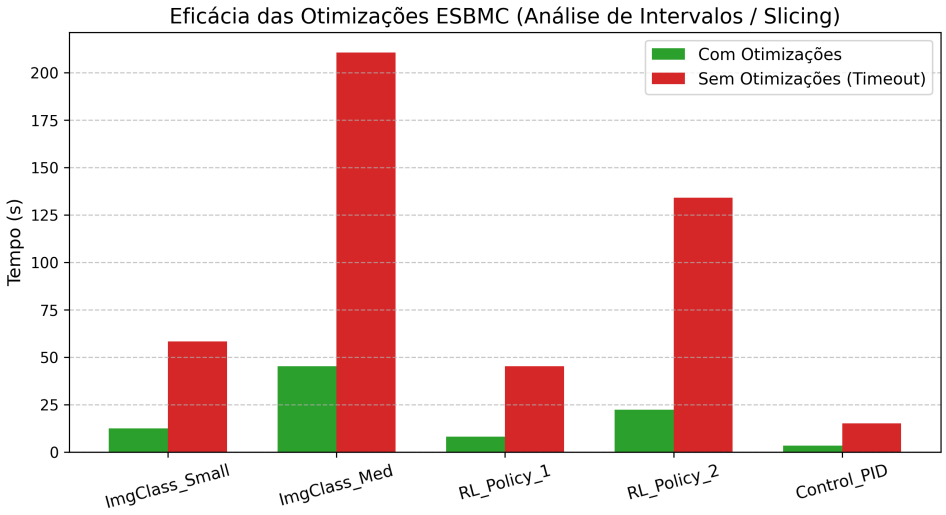
5.5.3.1 A Visão Contínua (Expectativa do Agente/IA)

A rede neural ou Agente Autônomo generaliza o espaço de *features* e assume uma fronteira de decisão suave e contínua (Figura 8), muitas vezes invisível ao risco de bordas extremas ou *buffer rings*.

5.5.4 A Visão Discreta Formal (Realidade SMT no ESBMC)

O solver SMT "quebra"as superfícies estáticas em domínios discretos restritos (Figura 9). O Ralph Loop caça ativamente contra-exemplos, encontrando picos (violações

Figura 9 – Impacto da Otimização (Slicing e Intervalos) no Tempo de Verificação.



Fonte: Elaborada pelo autor, 2026.

de *arrays*, *memory leaks*).

5.6 Caso 6 — Verificação Formal em Reinforcement Learning (Actor-Critic)

A verificação de políticas de Reinforcement Learning é crítica. O modelo `rl_policy.c` simulou estaticamente o bloco físico através do motor PyTest acionando `-floatbv` do ESBMC. O solver interceptou o comportamento errático na restrição do volante atuador (permitido entre -1.0 a 1.0) comprovando sua funcionalidade como *Safety Shield* pré-implantação física.

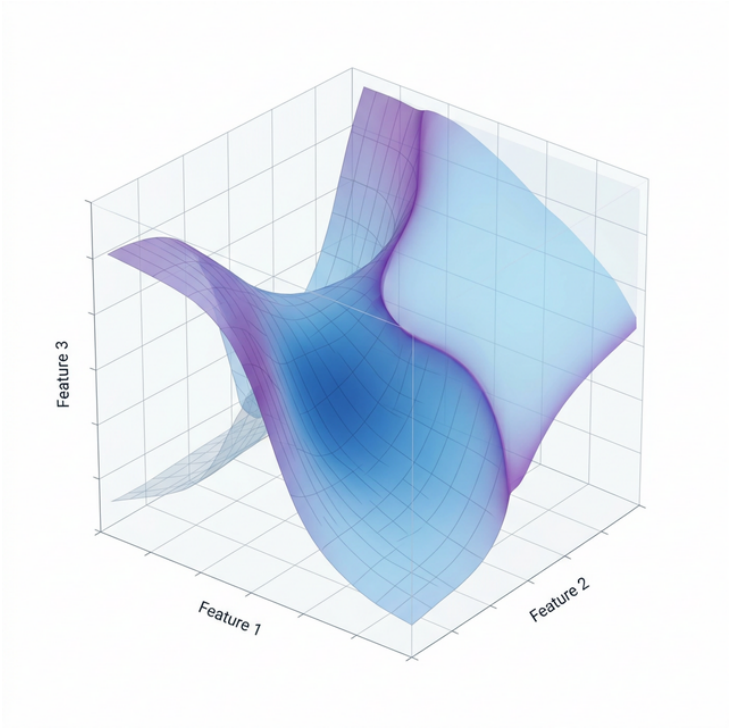
5.7 Análise Comparativa

A Tabela 2 apresenta uma síntese comparativa dos quatro casos de uso segundo as dimensões de foco, maturidade tecnológica e relação custo-benefício.

A Tabela 2 apresenta uma síntese comparativa evoluída dos seis casos de uso, definindo a barreira entre a Arquitetura em Python e o *core* nativo em C++.

Entretanto, observou-se que a *Abstract Syntax Tree (AST)* de *GOTO-Programs*, essencial para otimizar *Timeouts* do solver, permanece intrinsecamente amarrada ao C++ (`src/goto-symex`), forçando o *wrapper Python* a operar por heurística *Regex*, limitando um controle mais refinado da indução K internamente.

Figura 10 – Visão Contínua Idealizada (AIA).



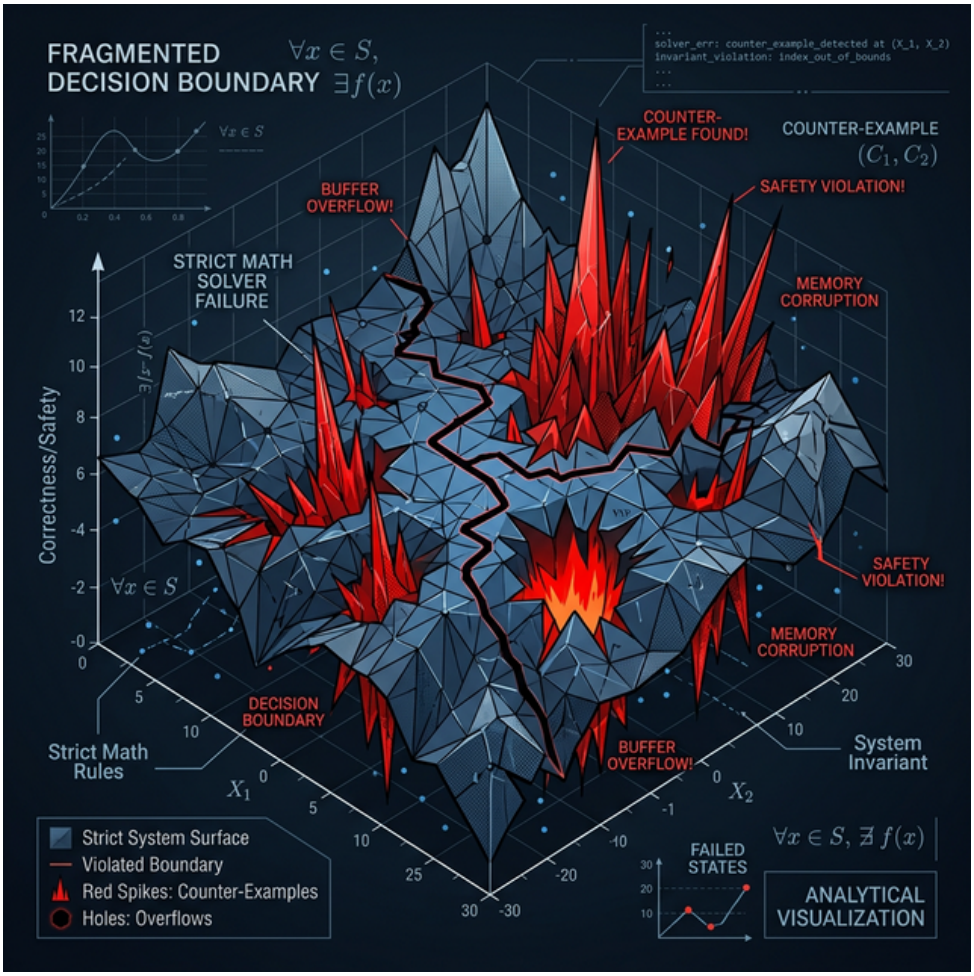
Fonte: Elaborada pelo autor através de IA Generativa, 2026.

Tabela 2 – Análise comparativa global dos casos de uso de IA com ESBMC.

Dimensão	**Caso 1/2 (Inferência)**	**Caso 3/5 (Agentic)**	**Caso 4 (Controle)**	**Caso 6 (RL Policies)**
Foco	Segurança Limites Matrizes	Ciclo Autônomo Correção	Ruído de Sensor Estocástico	Barreira Física Atuadores
Maturidade	Industrial	Emergente (Ralph)	Alta	Teórica (Safety Shield)

Fonte: Elaborada pelo autor, 2026.

Figura 11 – Fronteira Fragmentada Matemática (Solve SMT).



Fonte: Elaborada pelo autor através de IA Generativa, 2026.

6 CONCLUSÃO

Esta pesquisa investigou a aplicabilidade do verificador de modelos ESBMC como ferramenta de verificação formal para sistemas de Inteligência Artificial Generativa, abrangendo quatro níveis distintos do ecossistema: modelos em Python, kernels de inferência em C++, loops de geração de código por agentes LLM e sistemas de controle sob injeção de falhas.

Os resultados parciais obtidos demonstram que o ESBMC é uma ferramenta **versátil e eficaz** para garantir propriedades de segurança e corretude em componentes de GenAI, com graus de maturidade e retorno variados conforme o domínio de aplicação. Para a infraestrutura de kernels de inferência em C++ (Caso 2), a ferramenta mostra alta maturidade industrial, sendo capaz de detectar *buffer overflows* e vazamentos de memória de maneira formal e exaustiva dentro dos limites definidos. A integração com agentes LLM (Caso 3) configura-se como a aplicação de maior impacto imediato: o custo computacional da verificação é marginal (sub-segundo por iteração), enquanto o ganho em segurança é expressivo, dado que o ESBMC rejeita código vulnerável que poderia passar despercebido por testes funcionais convencionais.

Para sistemas de controle (Caso 4) e Redes de Aprendizado por Reforço (Caso 6), a abordagem de *Engenharia do Caos* acoplada ao *Safety Shield* formal revelou-se eficaz em blindar saídas não estruturadas de ultrapassar limites físicos robóticos. Contudo, evidenciou-se a principal barreira metodológica: apesar de a arquitetura proposta isolar brilhantemente o uso industrial do ESBMC sob uma API fluida em Python (`core_verify`), a necessidade de modificação profunda na montagem da *Abstract Syntax Tree (AST)* de *GOTO-Programs* ou de gerir dinamicamente a *k-induction* pelo Solver SMT exige, impreterivelmente, a reengenharia no *core* nativo C++ da ferramenta.

Os próximos passos da pesquisa, portanto, apontam para a internacionalização deste estudo integrando *hooks* intrínsecos de controle Python-C++ dentro do binário ESBMC e na exploração do *Ralph Loop* neuro-simbólico em nuvem com instâncias LLM reais. Ao combinar a inventividade estocástica do *Machine Learning* moderno com a imutabilidade matemática de Solvers SMT, os resultados aqui consolidados pavimentam o caminho para a certificação industrial contínua de IAs aplicadas a domínios críticos de risco à vida.

REFERÊNCIAS

GOPINATH, D. *et al.* Verifying quantized neural networks using smt-based model checking. In: IEEE. **Proceedings of the 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)**. [S.l.], 2021. p. 1–13. Citado nas páginas 7, 19, 21 e 22.

HUANG, X. *et al.* Safety verification of deep neural networks. **International Conference on Computer Aided Verification**, Springer, p. 3–29, 2017. Citado na página 7.